

UNIDAD N° 1

Introducción a los Sistemas Operativos

Sistemas Operativos · 2° Año ISI · UTN FRVM · 2026

Agenda

UNIDAD 1

01

Introducción

Definición, historia y generaciones

04

Conceptos del SO

Procesos, archivos, E/S, protección, shell

02

Hardware

CPU, memoria, discos, E/S, buses, arranque

05

Llamadas al Sistema

API POSIX vs Win32

03

Tipos de SO

Mainframe, servidores, móviles, embebidos...

06

Estructura del SO

Monolítico, capas, microkernel, VM

Componentes de una Computadora Actual

INTRODUCCIÓN



Uno o más procesadores



Memoria principal



Discos



Impresoras



Teclado / Mouse



Monitor



Interfaz de red



Dispositivos E/S

Sistema Operativo — Definición

DEFINICIÓN

Software que permite la abstracción de los recursos de Hardware a las aplicaciones y se encarga de la administración de los mismos.

Programa que controla la ejecución de los programas de aplicación y que actúa como interfaz entre las aplicaciones del usuario y el hardware del computador.

Top-down View

Proporciona abstracciones a programas y aplicaciones

Bottom-up View

Administra las piezas de un sistema complejo

Alternative View

Asignación ordenada de recursos entre programas

Sistema Operativo — Modos de Ejecución

DEFINICIÓN

Modo Usuario

- Acceso restringido al hardware
- Ejecuta aplicaciones del usuario
- Usa syscalls para pedir servicios al kernel
- Espacio de memoria protegido
- Sin acceso directo a dispositivos

syscall →

Modo Kernel

- Acceso total al hardware
- Ejecuta código del SO (kernel)
- Puede ejecutar cualquier instrucción
- Gestión de interrupciones
- Acceso directo a memoria física

Ubicación del Sistema Operativo

DEFINICIÓN

Web browser · Email · Music player (Aplicaciones)

Interfaz de usuario (GUI / Shell)

Sistema Operativo (Kernel)

Hardware

Modo
Usuario

**Modo
Kernel**

Capas y Vistas de un Sistema de Computación

DEFINICIÓN

Usuario Final

Programador

Programas de Aplicación

Utilidades

Sistema Operativo

Hardware del Computador

Diseñador
del SO

Sistema Operativo — Objetivos

DEFINICIÓN



Facilidad de uso

Ocultar la complejidad del hardware. Ofrecer interfaces amigables al usuario y al programador.



Eficiencia

Aprovechar al máximo los recursos de la máquina. Minimizar tiempos ociosos de CPU, memoria y E/S.



Capacidad de evolución

Permitir el desarrollo y prueba de nuevas funciones sin afectar el servicio. Adaptarse a nuevo hardware.

Sistema Operativo — Servicios

DEFINICIÓN

Desarrollo de programas

Editores, compiladores, depuradores

Acceso al sistema (login)

Autenticación de usuarios

Ejecución de programas

Carga en memoria, planificación de CPU

Detección y respuesta a errores

Hardware, software, operador

Acceso a dispositivos E/S

Abstracción de drivers

Contabilidad

Estadísticas de uso de recursos

Acceso a archivos

Crear, leer, escribir, borrar

SO como Máquina Extendida — Abstracciones

DEFINICIÓN

Hardware
(interfaz «fea»)

Sistema Operativo

Aplicaciones
(interfaz «limpia»)

Abstracciones que ofrece el SO

- Procesos** *Abstracción de la CPU*
- Espacio de dir.** *Abstracción de la memoria*
- Archivos** *Abstracción del almacenamiento*
- E/S** *Abstracción de dispositivos*
- Shell / GUI** *Abstracción de la interfaz de usuario*

SO como Administrador de Recursos

DEFINICIÓN

- Asignación ordenada de recursos entre las diferentes aplicaciones
- Multiplexación en espacio y tiempo de los recursos
 - Ejemplos: impresora, procesador, memoria, E/S, etc.

Top-down View

Proporciona abstracción a los programas y aplicaciones

Bottom-up View

Administra las piezas de un sistema complejo

Alternative View

Provee asignación ordenada de recursos entre distintos programas

Sistema Operativo — Historia

HISTORIA			
1ª Gen.	1945–1955	Tubos de vacío	Sin SO. Programación en lenguaje máquina / tarjetas perforadas.
2ª Gen.	1955–1965	Transistores	Procesamiento por lotes. Primeros SO (FMS, IBSYS).
3ª Gen.	1965–1980	Circuitos integrados	Multiprogramación, tiempo compartido. UNIX, CP/M.
4ª Gen.	1980–hoy	VLSI – PC	GUI, redes, Windows, Linux, macOS.
5ª Gen.	1990–hoy	Computadoras móviles	Android, iOS. Sistemas distribuidos y en la nube.

Procesamiento por Lotes — Flujo

HISTORIA

\$JOB → \$FORTRAN → \$LOAD → \$RUN → \$END

Formato típico de un trabajo (job) en procesamiento por lotes

1

Tarjetas perforadas

2

Cinta de entrada

3

Procesamiento

4

Cinta de salida

5

Impresión

Programador lleva tarjetas al operador

Tarjetas se leen y graban en cinta

Computadora lee la cinta y ejecuta

Resultados grabados en otra cinta

Cinta de salida enviada a impresora

Procesamiento por Lotes — Estructura de un Job

HISTORIA

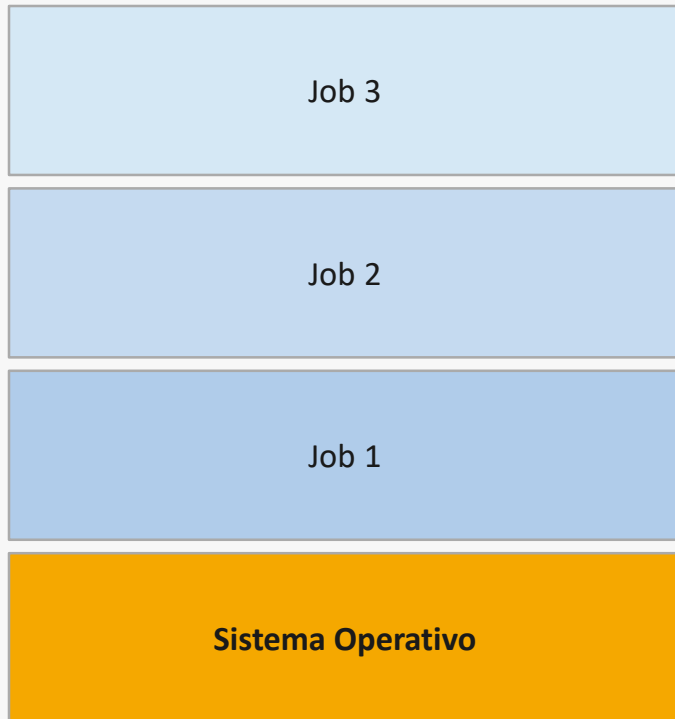


Las tarjetas de control (\$JOB, \$FORTRAN...) le indicaban al sistema qué hacer con cada trabajo.

Circuitos Integrados — Multiprogramación

HISTORIA

Particiones de memoria



¿Qué es la multiprogramación?

- Varios jobs cargados en memoria simultáneamente
- Cuando un job espera E/S, la CPU ejecuta otro
- Aumenta la utilización de la CPU
- El SO gestiona qué job corre en cada momento
- Base del time-sharing (tiempo compartido)

4ª y 5ª Generación — PC y Computación Móvil

HISTORIA

Computadoras Personales (1980–)

- Procesadores baratos (8086, 286...)
- GUI: Xerox PARC → Mac → Windows
- MS-DOS, Windows, OS/2, Linux
- Redes: TCP/IP en escritorio
- Interfaz amigable para no técnicos

Computación Móvil (1990–)

- Smartphones y tablets ubícuos
- Android (Linux) e iOS (Darwin/XNU)
- ARM: bajo consumo de energía
- Apps distribuidas (App Store, Play)
- SO en tiempo real + gestión de batería

Principales Logros del Desarrollo de los SO

HISTORIA

Procesos

Abstracción de ejecución concurrente. Permite multiprogramación y tiempo compartido.

Gestión de memoria

Memoria virtual, paginación, segmentación. Protección entre procesos.

Protección y seguridad

Modos usuario/kernel, llamadas al sistema controladas, control de acceso.

Planificación y gestión

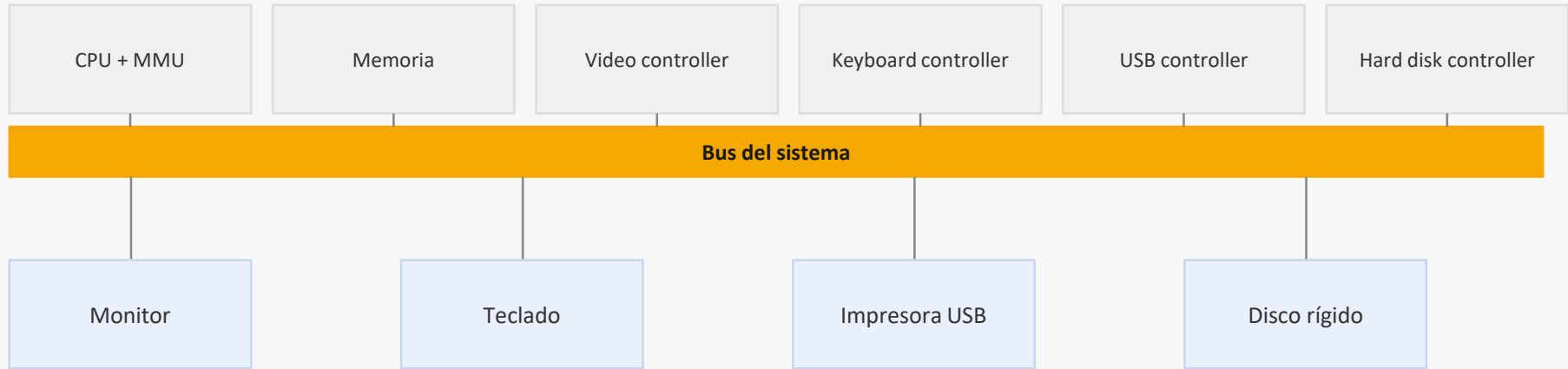
Algoritmos de CPU scheduling. Gestión justa y eficiente de recursos escasos.

Estructura del sistema

Microkernel, capas, módulos. Diseño que facilita mantenimiento y evolución.

Hardware de la Computadora — Revisión

HARDWARE



Cada componente tiene un controlador que se comunica con la CPU a través del bus.

Procesador (CPU)

HARDWARE

- ▶ **«Cerebro» de la computadora**
- ▶ **Ciclo de instrucción:**
 - ▶ Fetch — obtener instrucción de memoria
 - ▶ Decode — decodificar tipo y operandos
 - ▶ Execute — ejecutar la instrucción
- ▶ **Registros internos (rápidos, en el chip):**
 - ▶ Contador de programa (PC) — dirección próxima instrucción
 - ▶ Apuntador de pila (SP) — tope del stack
 - ▶ Registro de estado (PSW) — flags de condición
 - ▶ Registros generales — operandos y resultados

Registros del CPU

PC

SP

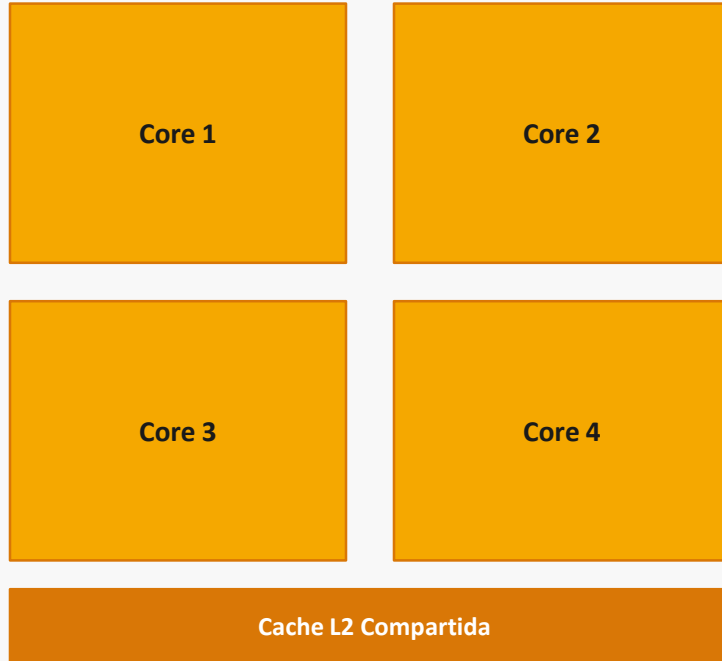
PSW

R0–Rn

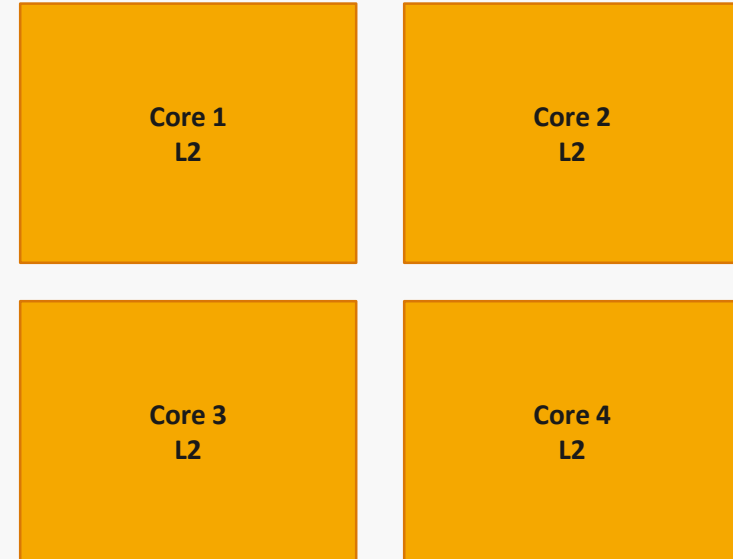
Procesadores — Múltiples Núcleos (Cores)

HARDWARE

Cache L2 compartida



Cache L2 por core



Memoria — Jerarquía

HARDWARE

Registros ~1 ns < 1 KB

Caché (L1/L2) 2–5 ns 256 KB–8 MB

Memoria RAM ~10 ns 1–64 GB

Disco (SSD/HDD) ~1 ms 256 GB–20 TB

✓ Cache hit: dato en caché → rápido

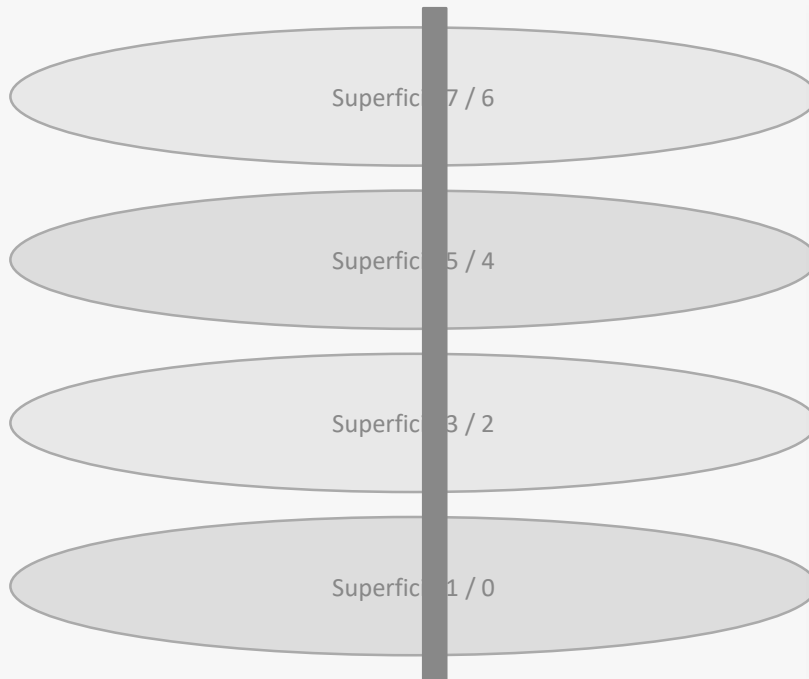
✗ Cache miss: buscar en RAM → más lento

▲ Más rápido / más caro / menor capacidad

Más lento / más barato / mayor capacidad ▼

Discos — Estructura Interna

HARDWARE



Conceptos clave

Platina (Platter): Disco magnético. Ambas caras utilizables.

Pista (Track): Círculo concéntrico. Cada cara tiene miles.

Sector: Arco de pista. Unidad mínima de lectura (512 B / 4 KB).

Cilindro: Misma pista en todas las platinas. Acceso sin mover brazo.

Acceso: Seek time + rotational latency + transfer time.

Cintas Magnéticas

HARDWARE

Acceso secuencial

Para leer el byte N hay que pasar por $0 \dots N-1$. No hay acceso aleatorio.

Bajo costo por GB

El medio más barato para almacenamiento masivo. Ideal para backup.

Alta capacidad

Cintas modernas LTO almacenan entre 18 TB y 150 TB por cartucho.

Uso actual

Backup de centros de datos, archivos fríos (cold storage), cloud offline.

Dispositivos de Entrada / Salida

HARDWARE

- ▶ **Dispositivos:** teclado, mouse, pantalla, disco, red, USB, etc.
- ▶ **Controladores físicos** (ej. IDE, SATA, USB): circuito de interfaz del dispositivo
- ▶ **Interrupciones:** señal al CPU cuando el dispositivo termina una operación
- ▶ **Driver en el SO:** software que sabe cómo hablar con el controlador
- ▶ **Formas de instalación:**
 - ▶ Dentro del kernel (compilado)
 - ▶ Módulo cargable en tiempo de ejecución
 - ▶ Plug and Play: el SO detecta y configura el dispositivo automáticamente

Pila de E/S

Aplicación

Llamada al sistema (syscall)

Subsistema de E/S del SO

Driver del dispositivo

Controlador (hardware)

Dispositivo físico

Buses — Interconexión de Componentes



Arranque de la Computadora

HARDWARE

1

BIOS / UEFI

Firmware en ROM. Ejecuta el POST (Power-On Self Test).

2

POST

Verifica RAM, teclado y buses. Detecta dispositivos de arranque.

3

Dispositivo de arranque

Selecciona disco, USB o red según configuración.

4

MBR / GPT

Se lee el primer sector del disco. Contiene el bootloader inicial.

5

Bootloader (GRUB/BCD)

Carga el kernel del SO en memoria RAM.

6

Kernel del SO

Se inicializa el sistema, monta el filesystem raíz, lanza init/systemd.

Tipos de Sistemas Operativos

TIPOS DE SO



Mainframe

z/OS



Servidores

Linux, Win Server



Multiprocesadores

SMP / NUMA



PC / Desktop

Windows, Linux



Móvil

Android, iOS



Embebido

Routers, IoT



Tiempo Real

VxWorks, FreeRTOS



Tarjetas int.

JavaCard

Conceptos del SO — Proceso (1)

CONCEPTOS

Proceso = programa en ejecución (concepto clave en todos los SO)

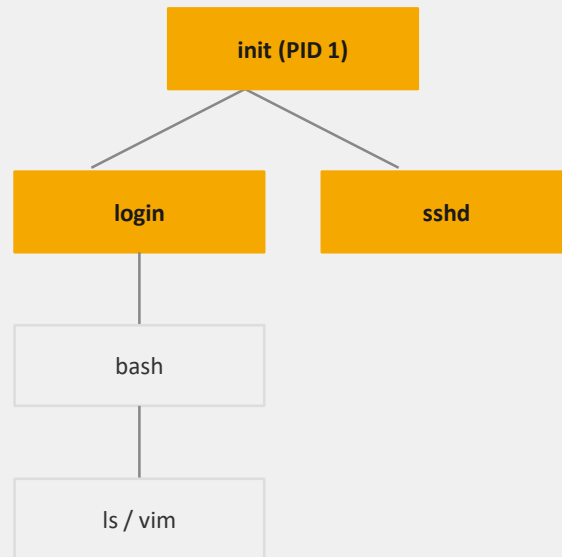
- ▶ Asociado a un espacio de memoria y un conjunto de recursos
- ▶ Puede ser visto como un contenedor
- ▶ Posee toda la información que necesita el programa para correr:
 - ▶ Código (text segment)
 - ▶ Datos (data segment / heap)
 - ▶ Stack — llamadas a funciones
 - ▶ Registros del CPU en el momento actual
 - ▶ Tabla de archivos abiertos

Conceptos del SO — Proceso (2)

CONCEPTOS

- ▶ Jerarquía de procesos — árbol padre/hijo (UNIX: fork)
- ▶ Creación: `fork()` + `exec()` en POSIX | `CreateProcess()` en Win32
- ▶ Tabla de procesos — BCP (Bloque de Control del Proceso)
- ▶ Intérpretes de comandos (shell) — también son procesos
- ▶ Comunicación entre procesos (IPC):
 - ▶ Señales (ej. `kill -9 <pid>`)
 - ▶ Tuberías (pipes) | sockets | memoria compartida
- ▶ Identificadores: **UID, GID** — superusuario (root/Administrator)

Jerarquía UNIX

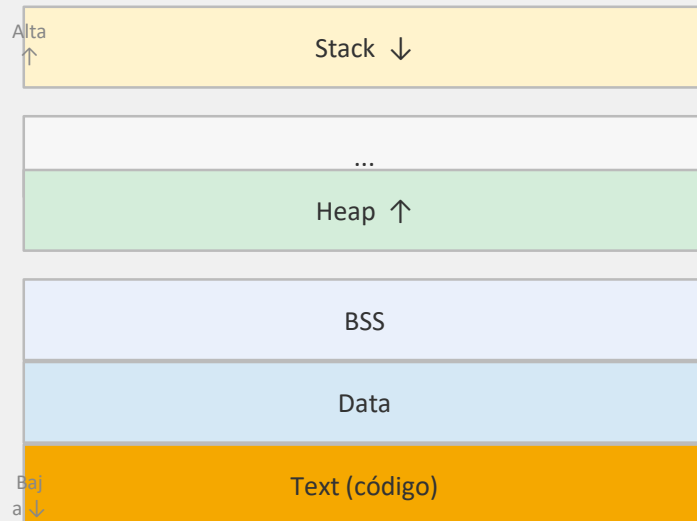


Conceptos del SO — Espacio de Direcciones

CONCEPTOS

- ▶ Administración de memoria — función central del SO
- ▶ Espacio de direcciones: rango de direcciones de memoria que un proceso puede usar
- ▶ Cada proceso cree tener su propia memoria (memoria virtual)
- ▶ El SO mapea direcciones virtuales → físicas mediante la MMU (Memory Management Unit)

Layout típico de proceso



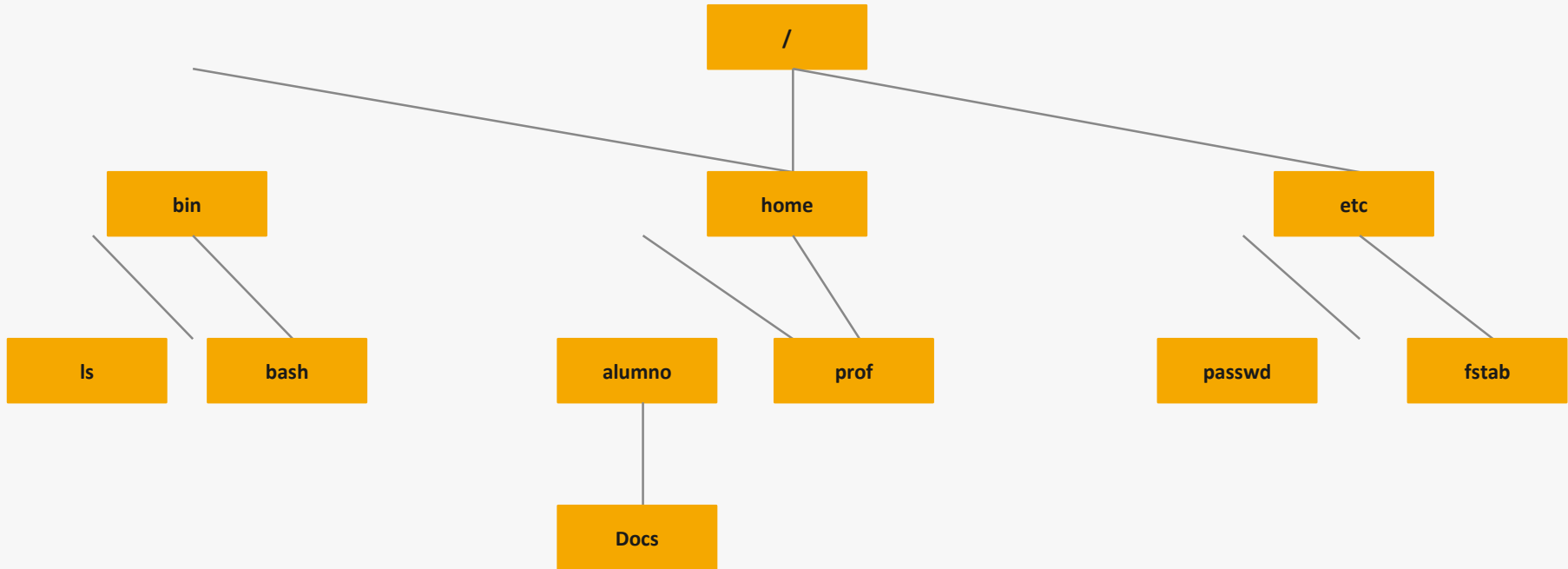
Conceptos del SO — Archivos (1)

CONCEPTOS

- ▶ Jerarquía de directorios — árbol (root / en UNIX, C:\ en Windows)
- ▶ Ruta absoluta: desde la raíz (/usr/bin/bash)
- ▶ Ruta relativa: desde el directorio de trabajo (../config)
- ▶ Directorio de trabajo (cwd) — contexto del proceso
- ▶ Descriptor de archivo — entero que identifica un archivo abierto
- ▶ Tipos especiales:
 - ▶ Archivos de bloque — discos (acceso por bloques)
 - ▶ Archivos de carácter — teclado, terminal (acceso byte a byte)
 - ▶ Tuberías (pipes) — comunicación entre procesos
- ▶ Sistema de archivo raíz: / | Montar y desmontar (mount/umount)

Conceptos del SO — Archivos: Jerarquía

CONCEPTOS

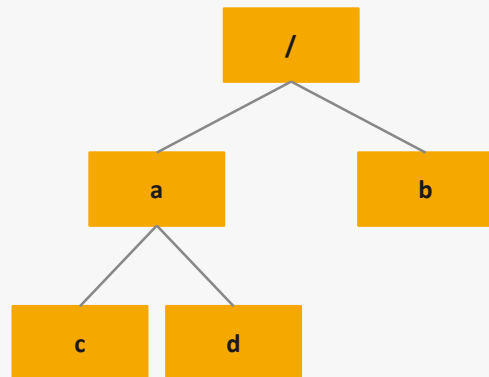


Estructura de directorios: cada nodo amarillo es un directorio, las hojas son archivos.

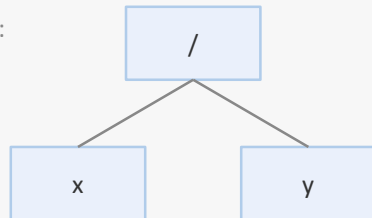
Conceptos del SO — Archivos: Montar / Desmontar

CONCEPTOS

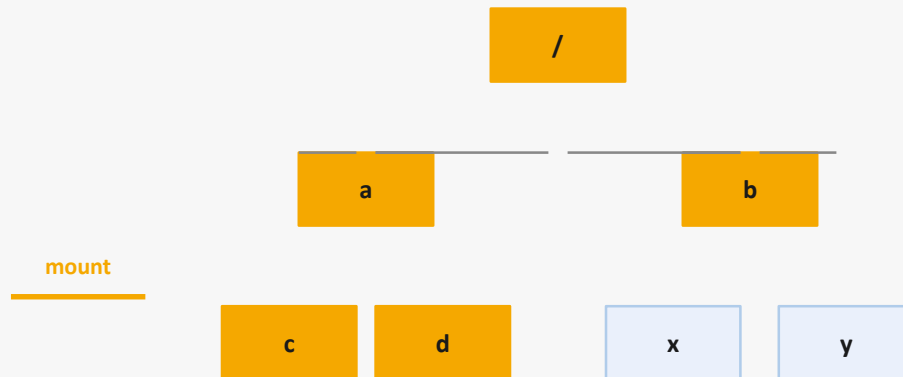
(a) Antes de montar



CD-ROM:



(b) Después de montar



Conceptos del SO — Archivos: Tuberías (Pipes)

CONCEPTOS

- La salida estándar (stdout) del proceso A se convierte en la entrada estándar (stdin) del proceso B
- Comunicación unidireccional y en memoria (sin archivos intermedios)
- Primitiva fundamental de IPC en sistemas POSIX



```
$ ls -la | grep .txt | sort
```

Conceptos del SO — Entrada / Salida

CONCEPTOS

- ▶ El SO tiene un subsistema de E/S que es independiente del tipo de dispositivo
- ▶ La funcionalidad del driver depende del dispositivo; el subsistema de E/S, no
- ▶ Técnicas de E/S:
 - ▶ Polling (busy wait): CPU consulta continuamente si el dispositivo terminó
 - ▶ Interrupciones: el dispositivo avisa a la CPU al terminar (más eficiente)
 - ▶ DMA (Direct Memory Access): el dispositivo transfiere datos sin la CPU
- ▶ Organización del subsistema de E/S del SO:
 - ▶ Capa independiente del dispositivo (buffers, caché, scheduling)
 - ▶ Drivers — capa dependiente del dispositivo
 - ▶ Hardware interrupt handlers

Conceptos del SO — Protección

CONCEPTOS

Permisos de archivo

rwX para usuario, grupo y otros. Ej: `chmod 755 archivo`

Contraseñas

Autenticación de usuarios al iniciar sesión.

Confiabilidad

Acceso solo a recursos autorizados.

Integridad

El SO evita que procesos dañen datos de otros procesos.

Autenticidad

Verificar que el software proviene de fuentes confiables.

Formas de acceso

Lectura, escritura, ejecución, append, delete.

Antivirus / IDS

Herramientas externas al SO que agregan capas de protección.

Conceptos del SO — Shell

CONCEPTOS

- ▶ El shell es un programa (proceso) que interpreta comandos del usuario
- ▶ Ejemplos POSIX: `bash`, `sh`, `zsh`, `ksh`, `fish`, `dash`
- ▶ Windows: `CMD (cmd.exe)` y `PowerShell`
- ▶ Funciones principales:
 - ▶ Entrada de comandos → parámetros → salida (`stdout` / `stderr`)
 - ▶ Redirección: `>` `<` `>>` `2>` `|`
 - ▶ Variables de entorno (`PATH`, `HOME`...)
 - ▶ Scripts de automatización
- ▶ GUIs (interfaces gráficas) — también son shells, con ventanas en lugar de comandos

Ejemplos de uso

```
$ ls -la /etc
```

```
$ ps aux | grep nginx
```

```
$ cat /etc/passwd > backup.txt
```

```
$ chmod 755 script.sh
```

```
$ ./script.sh 2>&1 | tee log.txt
```

Conceptos del SO — Reflexiones

CONCEPTOS

Cada nueva especie de computadora pasa por el desarrollo de sus ancestros

Los SO embebidos de hoy repiten conceptos de los mainframes de los 60.

La tecnología «va y viene»

Mainframes → PCs → servidores → cloud → edge. Los conceptos reaparecen.

Conceptos obsoletos se traen a la actualidad frecuentemente

Tiempo compartido (60s) → containers modernos. Batch jobs → serverless.

Los cambios en la tecnología hacen que vuelvan esos conceptos

El procesamiento en la nube recuerda al modelo cliente/servidor centralizado.

Esto ocurre tanto para hardware como para software

ARM resurge en laptops y servidores. Terminales tontos → thin clients → web apps.

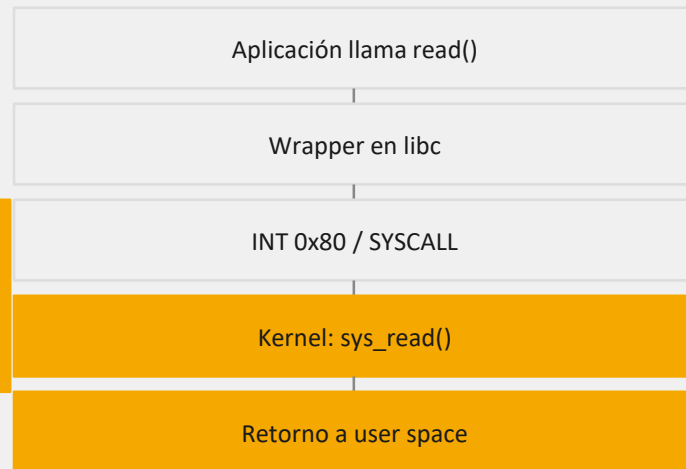
Llamadas al Sistema — Definición

SYSCALLS

Mecanismo por el cual un proceso en modo usuario solicita un servicio al kernel del SO.

- ▶ Es la única forma legal de que el código de usuario acceda a recursos del kernel
- ▶ **Grupos principales:**
 - ▶ Gestión de procesos: fork, exec, exit, waitpid
 - ▶ Gestión de archivos: open, read, write, close, seek, stat
 - ▶ Gestión de directorios: mkdir, rmdir, link, unlink, mount
 - ▶ Misceláneas: chmod, kill, time, chdir

Flujo de una syscall



Llamadas al Sistema — Gestión de Procesos

SYSCALLS

Llamada	Descripción
<code>pid = fork()</code>	Crea un proceso hijo idéntico al padre
<code>pid = waitpid(pid, &stat, opt)</code>	Espera que un hijo termine
<code>s = execve(name, argv, envp)</code>	Reemplaza la imagen del proceso
<code>exit(status)</code>	Termina el proceso y retorna estado

Ejemplo fork + exec:

```
pid = fork();  if (pid == 0) { execve("/bin/ls", argv, envp); }  else { waitpid(pid, &status, 0); }
```


Llamadas al Sistema — Gestión de Archivos

SYSCALLS

Llamada	Descripción
<code>fd = open(file, flags, mode)</code>	Abre un archivo para lectura, escritura o ambos
<code>s = close(fd)</code>	Cierra un archivo abierto
<code>n = read(fd, buffer, nbytes)</code>	Lee datos de un archivo a un buffer
<code>n = write(fd, buffer, nbytes)</code>	Escribe datos de un buffer a un archivo
<code>pos = lseek(fd, offset, whence)</code>	Mueve el puntero del archivo
<code>s = stat(name, &buf)</code>	Obtiene información del archivo (tamaño, permisos...)

Llamadas al Sistema — Gestión de Directorios

SYSCALLS

Llamada	Descripción
<code>s = mkdir(name, mode)</code>	Crea un nuevo directorio
<code>s = rmdir(name)</code>	Elimina un directorio vacío
<code>s = link(name1, name2)</code>	Crea name2 apuntando al mismo inodo que name1
<code>s = unlink(name)</code>	Elimina una entrada de directorio
<code>s = mount(special, name, flag)</code>	Monta un sistema de archivos
<code>s = umount(special)</code>	Desmonta un sistema de archivos

Llamadas al Sistema — Misceláneas

SYSCALLS

Llamada	Descripción
<code>s = chdir(dirname)</code>	Cambia el directorio de trabajo actual
<code>s = chmod(name, mode)</code>	Cambia los bits de protección de un archivo
<code>s = kill(pid, signal)</code>	Envía una señal a un proceso
<code>seconds = time(&secs)</code>	Obtiene el tiempo transcurrido desde 01/01/1970 (Unix epoch)

Las syscalls son la única forma legal de que el código de usuario acceda a recursos del kernel del SO

Llamadas al Sistema — Directorios: Ejemplo

SYSCALLS

Directorio /usr/ast

16	mail
81	games
40	test

Directorio /usr/jim

31	bin
70	memo
59	f.c
38	prog1

```
$ link("/usr/jim/memo", "/usr/ast/note") → crea entrada 70/note en /usr/ast
```

Después del link, el inodo 70 queda referenciado desde ambos directorios (hard link).

Llamadas al Sistema — API Win32 vs POSIX

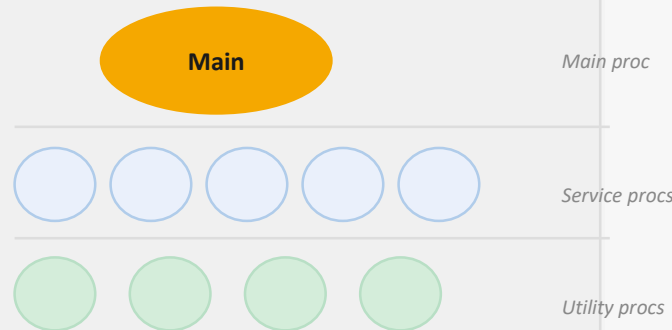
SYSCALLS		
POSIX (Linux)	Win32 (Windows)	Descripción
fork	CreateProcess	Crear proceso nuevo
waitpid	WaitForSingleObject	Esperar que un proceso termine
execve	(CreateProcess)	CreateProcess = fork + exec
exit	ExitProcess	Terminar ejecución
open	CreateFile	Crear/abrir un archivo
close	CloseHandle	Cerrar un archivo
read	ReadFile	Leer datos de un archivo
write	WriteFile	Escribir datos a un archivo
chmod	(none)	Win32 no soporta chmod (NT: SetSecurity)
mount	(none)	Win32 no soporta mount

Estructura del SO — Sistema Monolítico

ESTRUCTURA

- ▶ Todo el SO corre en modo kernel como un único programa
- ▶ Organización interna (Tanenbaum):
 - ▶ 1. Programa principal — invoca el procedimiento de servicio solicitado
 - ▶ 2. Procedimientos de servicio — ejecutan las llamadas al sistema
 - ▶ 3. Procedimientos utilitarios — ayudan a los procedimientos de servicio

Diagrama monolítico



✓ Alto rendimiento — llamadas directas sin overhead de IPC

✗ Un bug en cualquier parte puede derribar todo el sistema

Estructura del SO — Sistemas de Capas

ESTRUCTURA

5

El operador

4

Programas de usuario

3

Gestión de Entrada / Salida

2

Comunicación operador-proceso

1

Gestión de memoria y drum

✓ Modular: cada capa solo usa servicios de la inferior

n

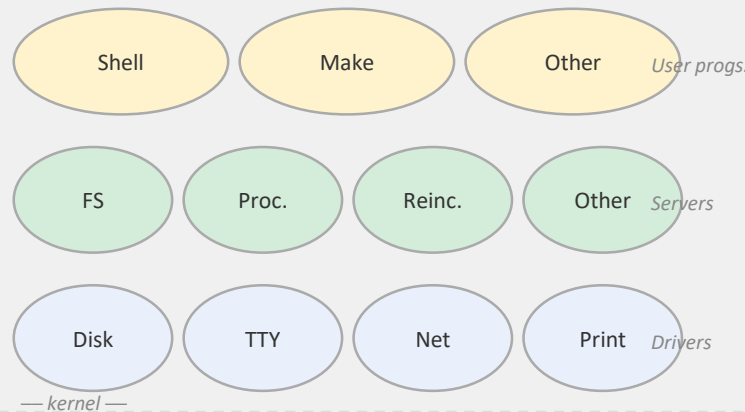
✗ Overhead por múltiples niveles de indirección

Estructura del SO — Microkernel

ESTRUCTURA

- ▶ Kernel mínimo: solo IPC, planificación básica y gestión de memoria
- ▶ El resto corre en modo usuario como servidores:
 - ▶ Servidor de sistema de archivos
 - ▶ Servidor de procesos (reencarnación)
 - ▶ Drivers de dispositivos
- ▶ Ejemplos: MINIX 3, QNX, Mach, L4, seL4

Arquitectura Microkernel



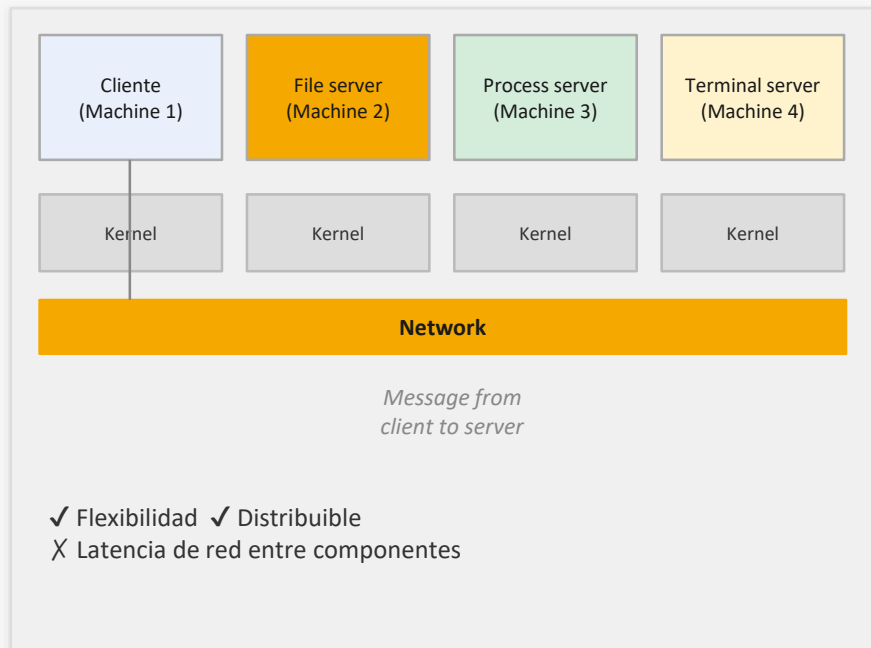
✓ Robusto: un driver que falla no tumba el kernel

✗ Overhead por mensajes IPC entre servidores

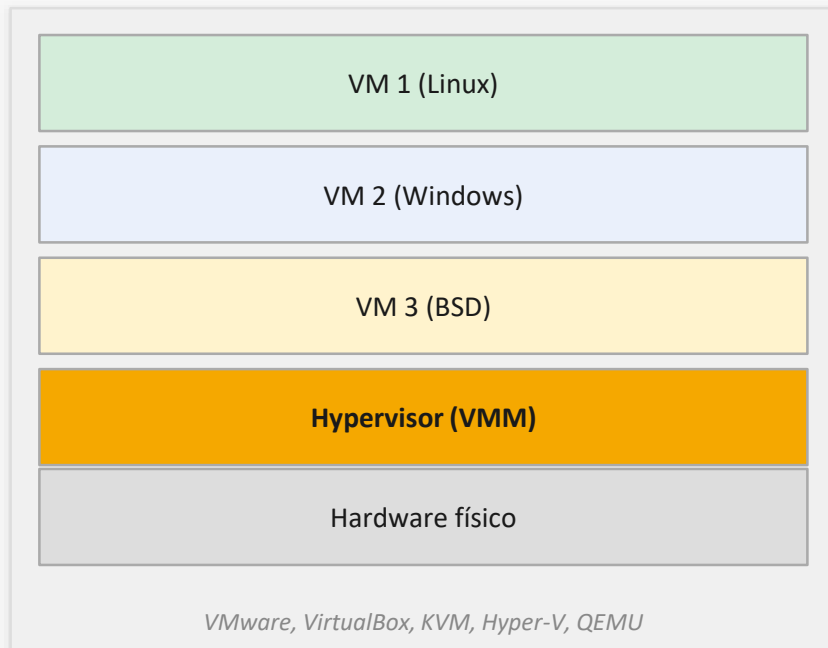
Estructura del SO — Cliente/Servidor y Máquinas Virtuales

ESTRUCTURA

Cliente / Servidor



Máquinas Virtuales



Resumen — Unidad 1

Resumen

- 1 El SO es administrador de recursos y máquina extendida — define las syscalls
- 2 Evolucionó desde tubos de vacío hasta contenedores, cloud e IoT
- 3 El hardware (CPU, memoria, disco, E/S, buses) es lo que el SO abstrae y gestiona
- 4 Los procesos, archivos, E/S, protección y shell son las abstracciones clave
- 5 Estructuras: monolítico (Linux), microkernel (MINIX), capas, VM (VirtualBox/KVM)